



WEB AUTOMATION TESTING

CSC15003 - Kiểm thử phần mềm

Group 23KTPM2_02

SUT: EShop

Traditional & AI-Augmented Testing



testim

VÌ SAO CẦN WEB AUTOMATION TESTING?



- Để thay thế hoàn toàn Manual testing
- Để tìm ra các bug mới
- Để tiết kiệm chi phí ngay lập tức



- Manual testing linh hoạt nhưng tốn thời gian khi chạy regression lặp lại
- Automation tăng tính lặp lại, rút ngắn feedback loop và hỗ trợ CI/CD
- Cung cấp bằng chứng kiểm thử (report, screenshot, trace)
- Hệ thống doanh nghiệp phụ thuộc nhiều vào luồng nghiệp vụ web
- Web automation testing là nền tảng cho kiểm thử hệ thống hiện đại
- Automated testing phát hiện lỗi nhanh hơn manual testing
- Test xanh không đảm bảo phần mềm đúng — oracle phải chính xác



MỤC TIÊU & PHẠM VI



✓ 01. Phạm vi

- Chỉ tập trung kiểm thử tự động trên trình duyệt web.
- Hệ thống SUT sử dụng: EShop.

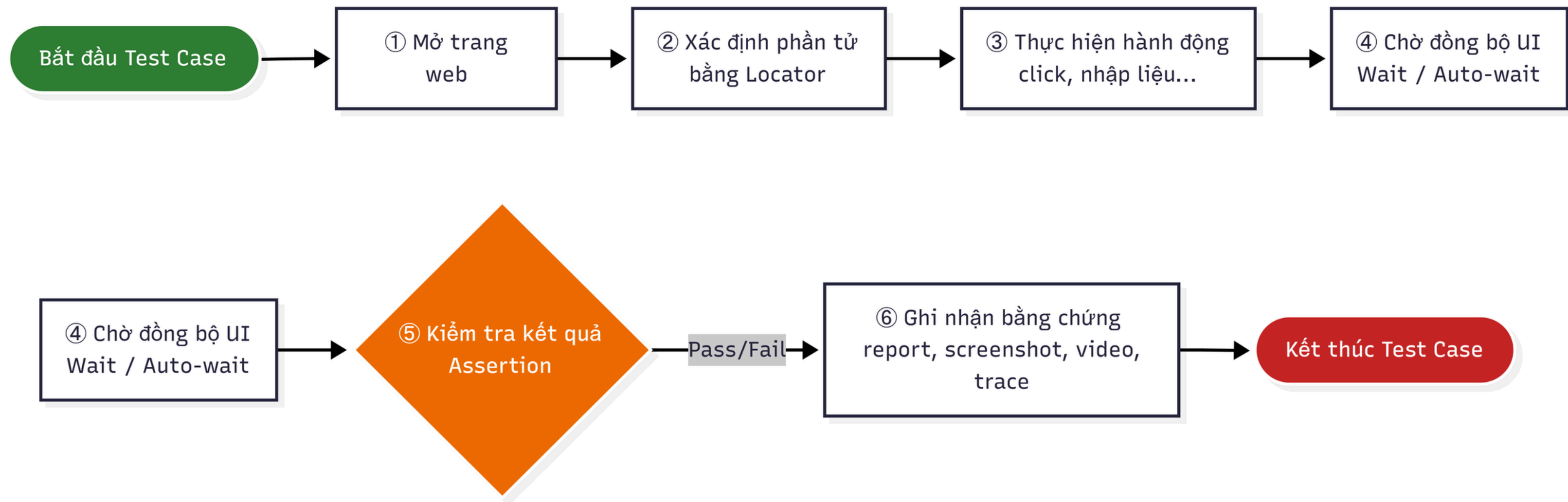
✓ 02. Mục tiêu

- Giới thiệu lý thuyết
- Khảo sát công cụ truyền thống
- Khám phá hướng AI-Augmented Testing

✓ 03. Hoạt động & Khuyến nghị:

- Demo thực tế trên EShop SUT.
- Thực hành Locator Brawl so sánh locator thủ công và AI
- Đưa ra khuyến nghị chọn công cụ phù hợp cho nhóm phát triển web.

WORKFLOW CƠ BẢN CỦA WEB AUTOMATION



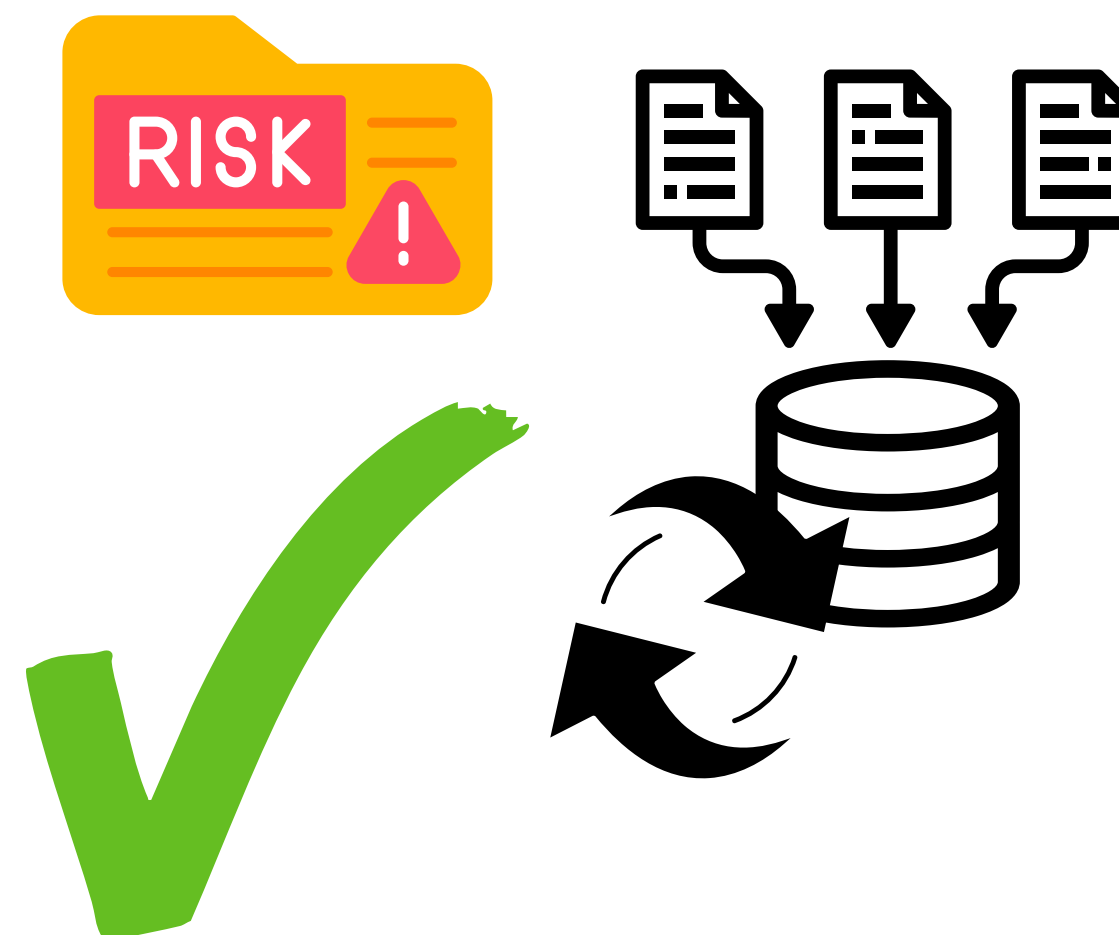
Các khái niệm then chốt:

- Locator | - Wait mechanism | - Assertion & Test Oracle
- Test Data/Fixture | - Test Isolation | - Report/Evidence

KHI NÀO NÊN TỰ ĐỘNG HÓA?

Nên tự động hóa khi...

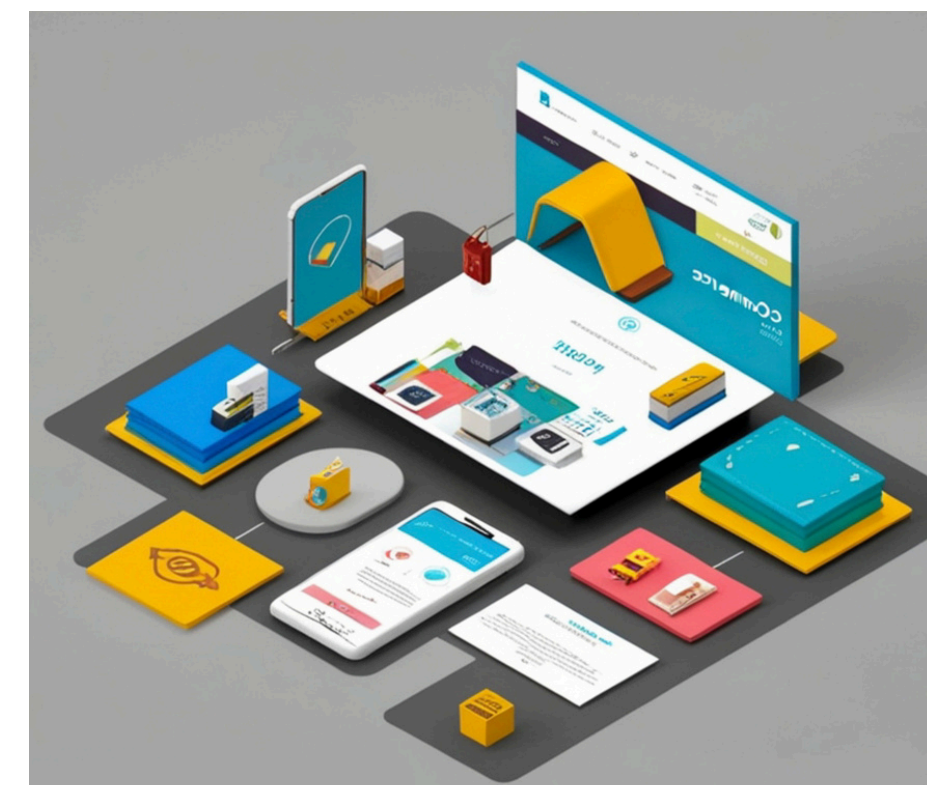
- Flow được chạy lặp lại trong regression
- Giá trị nghiệp vụ hoặc rủi ro cao
- Kết quả kỳ vọng rõ ràng, xác định được
- Dữ liệu test có thể chuẩn bị và reset được
- Giao diện tương đối ổn định, ít thay đổi



Không nên tự động hóa khi...

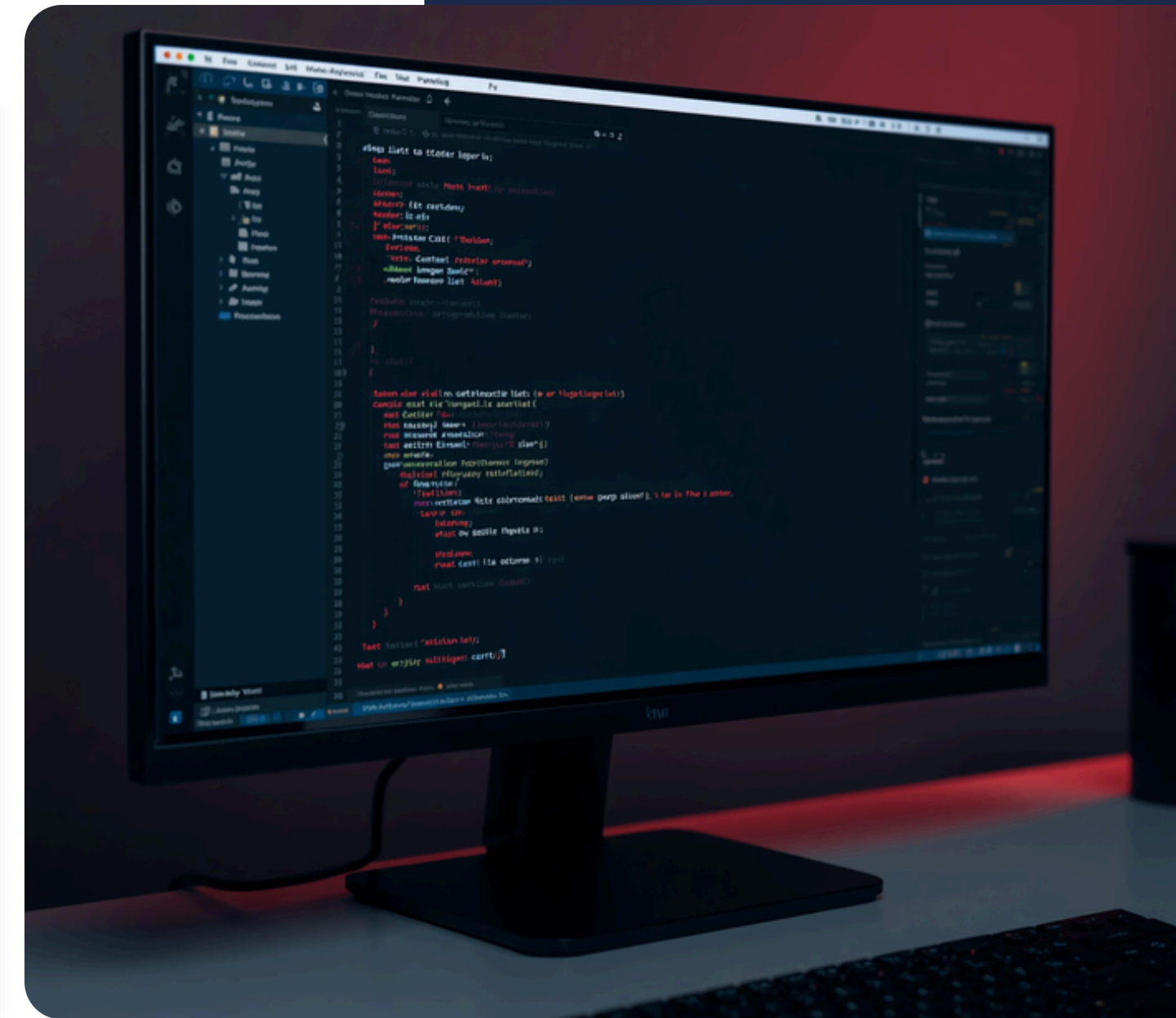


- Test chỉ chạy một lần duy nhất
- Cần đánh giá UX mang tính chủ quan
- Kết quả kỳ vọng chưa rõ ràng
- Phụ thuộc vào dịch vụ ngoài không kiểm soát được
- Giao diện thay đổi liên tục, không ổn định



RỦI RO THƯỜNG GẶP TRONG WEB AUTOMATION

- Flaky tests: timing, network, async UI, shared data
- Brittle locators: phụ thuộc sâu vào cấu trúc DOM
- False pass: assertion quá lỏng hoặc sai element (self-healing)
- False fail: assertion quá chặt hoặc dữ liệu không ổn định
- Slow tests: quá nhiều E2E, thiếu phân tầng kiểm thử
- Controls: ưu tiên role/label/text/test-id locators, dùng auto-wait
- Isolate test data; luôn review trace/log khi test thất bại



KHẢO SÁT CÔNG CỤ TRUYỀN THỐNG



01

Công cụ khảo sát: Playwright, Selenium, Cypress, Puppeteer.

Đánh giá theo các tiêu chí:

- Cài đặt & learning curve
- Kiến trúc & hỗ trợ trình duyệt
- Chiến lược locator & đồng bộ hóa
- Assertion
- Thực thi & báo cáo.

02

Tiêu chí mở rộng:

- Tích hợp CI/CD
- Khả năng bảo trì
- Hỗ trợ AI tooling.

Playwright dẫn đầu về auto-wait, modern locators và Trace Viewer.

Selenium phổ biến nhưng cần tích hợp thủ công nhiều thành phần.

QUYẾT ĐỊNH CHỌN CÔNG CỤ



- Công cụ chính: Playwright + TypeScript.

Lý do:

- Setup nhanh
- Locator hiện đại
- Auto-wait & web-first assertions giảm flaky test
- Trace Viewer hỗ trợ debug & evidence
- Hỗ trợ Chromium/Firefox/WebKit
- Chạy song song & tích hợp CI/CD
- Có AI Test Agents chính thức (Planner, Generator, Healer).



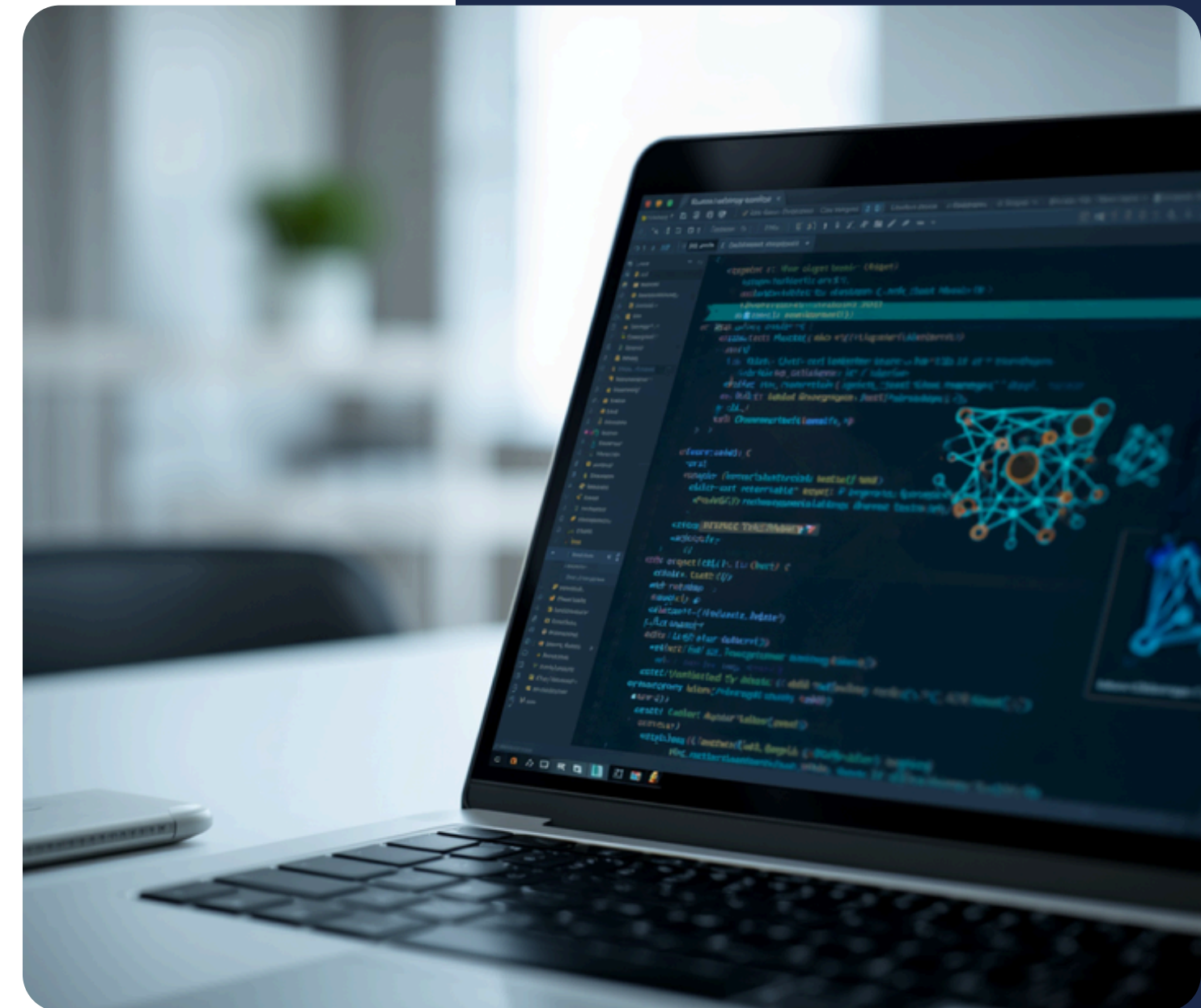
- Công cụ dự phòng: Cypress.

Lý do:

- Phù hợp cho team đã quen hệ sinh thái JavaScript
- Dashboard trực quan
- Dễ onboard cho dự án frontend nhỏ và mid-size.
- Sử dụng khi cần kiểm thử component-level hoặc khi team ưu tiên Cypress workflow.

AI TRONG KIỂM THỬ TỰ ĐỘNG

- AI-assisted: gợi ý code, locator, assertion
- AI-generated: sinh test script từ mô tả yêu cầu
- AI-healing: tự sửa locator khi UI thay đổi
- AI-agent: tự lập kế hoạch, sinh test, chạy và sửa lỗi
- Công cụ: Mabl, Testim AI, Katalon Studio, GitHub Copilot, Claude Code, OpenAI Codex
- AI tăng tốc viết và bảo trì test
- Con người vẫn phải định nghĩa yêu cầu, test oracle và review output của AI



HƯỚNG AI ĐƯỢC CHỌN CHO DEMO

AI Coding Assistant với Playwright

Hướng tiếp cận demo: Sử dụng AI coding assistant kết hợp Playwright để tạo và kiểm thử test script.

- Công cụ sử dụng: GitHub Copilot, Claude Code, OpenAI Codex.
- Quy trình thực hiện:
 1. Human viết scenario và mô tả yêu cầu
 2. AI sinh test draft từ mô tả
 3. Human kiểm tra locator và assertion
 4. Chạy test và đánh giá kết quả

Lý do không dùng Mabl/Testim/Katalon cho live demo: phụ thuộc SaaS license hoặc cloud, không tích hợp trực tiếp với Playwright codebase. Self healing phù hợp phân tích lý thuyết hơn là live demo.



DEMO FLOWS

Flow được chọn

- Login + Lockout: input flow kiểm tra khóa tài khoản sau nhiều lần đăng nhập sai — phù hợp kiểm tra state management
- Add-to-Cart: DOM động, phù hợp phân tích flakiness và so sánh locator strategies
- Checkout: E2E flow rủi ro cao, liên quan đến payment và server-side logic — cần oracle mạnh

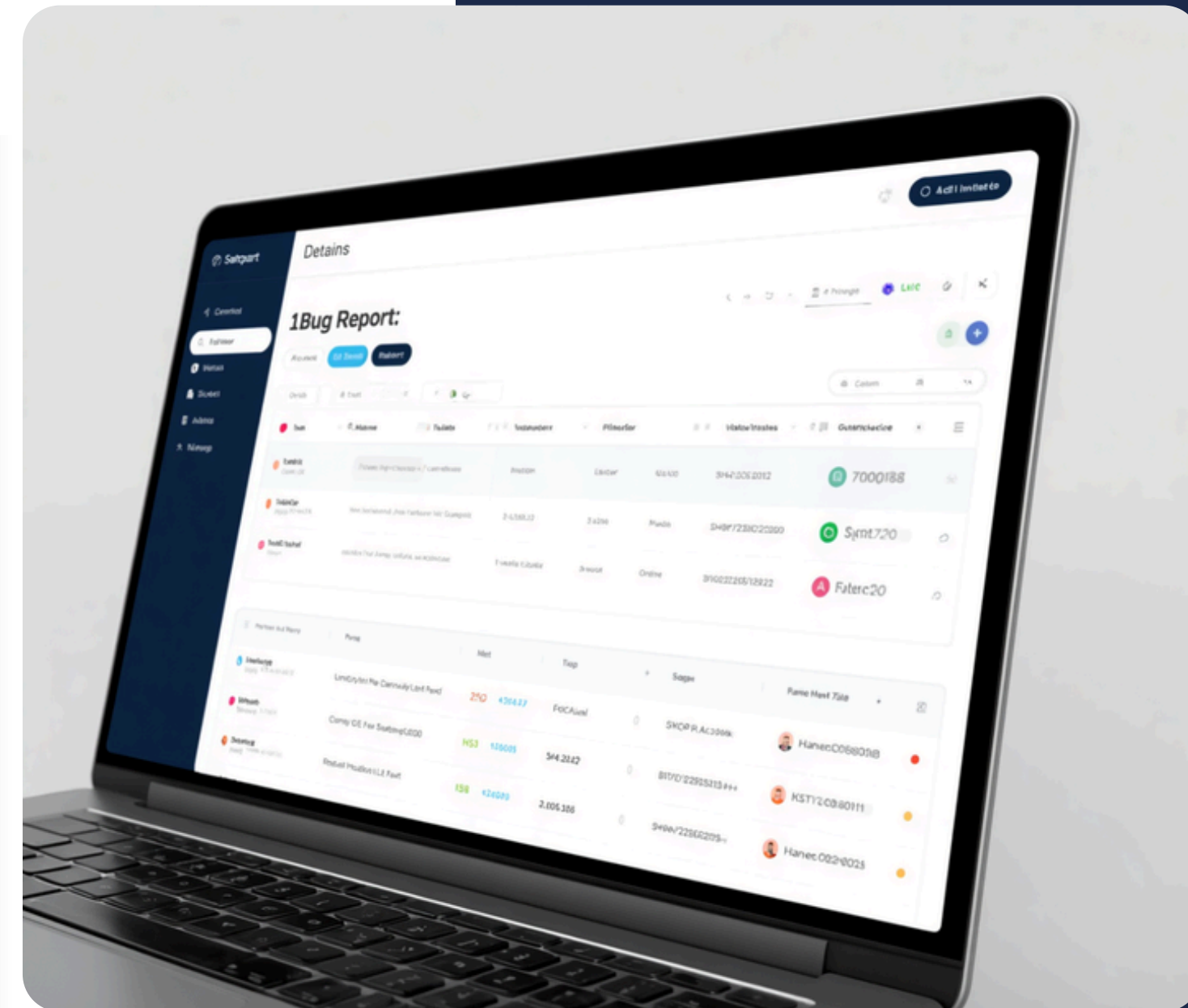


DEMO PLAYWRIGHT VÀ CÁC KẾT QUẢ



CÁC PHÁT HIỆN QUAN TRỌNG

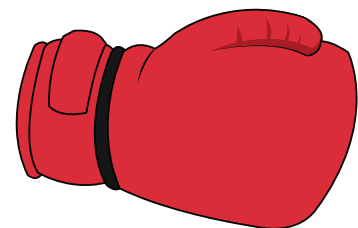
- Password field type="text" — lộ mật khẩu người dùng
- Login counter tăng sai — không đúng logic đếm
- Lockout 180 giây thay vì 30 giây theo spec
- Thêm cùng sản phẩm tạo 2 dòng, không gộp số lượng
- Navbar giỏ hàng thiếu badge số lượng
- Giỏ hàng không xóa sau khi checkout thành công
- Backend nhận client total_amount → mua giá thấp tùy ý



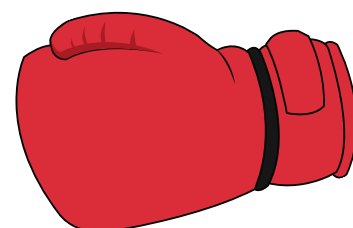
ACTIVITY, FAILURE MODES & KHUYẾN NGHỊ

- Locator Brawl: Team A dùng locator thủ công (Playwright), Team B dùng AI-suggested locators — cùng test Add-to-Cart, so sánh độ ổn định & khả năng bảo trì.
- Failure Modes: Self-healing có thể che giấu bug thật; AI-generated assertions thường quá lỏng; AI locators dễ gãy khi DOM thay đổi.
- Khuyến nghị: Dùng Playwright TypeScript làm framework E2E chính. Ưu tiên locator role/label/text/test-id. Dùng AI để tăng tốc viết draft, không dùng làm quyết định cuối. Luôn review locator & assertion trước khi merge. Theo dõi flaky rate của AI-generated tests sau 4–6 tuần.

ACTIVITY



Locator Brawl



Team A dùng locator thủ công
(Playwright)



Team B dùng AI-suggested locators

Cùng test Add-to-Cart, so sánh độ ổn định & khả năng bảo trì.



FAILURE MODES & KHUYẾN NGHỊ

- Failure Modes: Self-healing có thể che giấu bug thật; AI-generated assertions thường quá lỏng; AI locators dễ gãy khi DOM thay đổi.
- Khuyến nghị: Dùng Playwright TypeScript làm framework E2E chính. Ưu tiên locator role/label/text/test-id. Dùng AI để tăng tốc viết draft, không dùng làm quyết định cuối. Luôn review locator & assertion trước khi merge. Theo dõi flaky rate của AI-generated tests sau 4–6 tuần.